

Rapport de Projet : Gestion d'un Réseau de Métro

1. Introduction

Ce projet implémente un système de gestion d'un réseau de métro simplifié comprenant les lignes 1, 2, 3, 4, 6 et 14 du métro parisien. Le programme permet de rechercher des stations, lister les voisins, calculer des chemins optimaux et trier les stations par degré.

2. Structures de données

2.1 Structure Edge (Arête)

```
typedef struct Edge {
    int destination;    // ID de la station de destination
    int time;           // Temps de trajet en minutes
    struct Edge *next;  // Pointeur vers l'arête suivante
} Edge;
```

Justification : Une liste chaînée permet d'ajouter dynamiquement des voisins sans connaître leur nombre à l'avance. Chaque nœud stocke la destination et le temps de trajet, facilitant l'implémentation de Dijkstra.

2.2 Structure Station

```
typedef struct {
    int id_station;    // Identifiant unique de la station
    int degree;        // Nombre de voisins (degré sortant)
    Edge *edges;       // Liste chaînée des arêtes sortantes
} Station;
```

Justification : Cette structure combine :

- Un tableau de stations pour un accès rapide en $O(1)$ par index
- Des listes d'adjacence (via `edges`) pour un stockage efficace du graphe

Schéma de la représentation :

Tableau de stations

[0]: id=0, deg=1	————→ Edge (1, 4min) → NULL
[1]: id=1, deg=2	————→ Edge (2, 3min) → Edge (0, 4min) → NULL
[2]: id=2, deg=2	————→ Edge (3, 2min) → Edge (1, 3min) → NULL

Avantages de cette approche :

- Parcours des voisins en $O(\text{degree})$ au lieu de $O(n^2)$
- Mémoire proportionnelle au nombre d'arêtes réelles
- Ajout/suppression d'arêtes efficace

3. Algorithmes de tri

3.1 Algorithmes implémentés

Cinq algorithmes de tri ont été implémentés pour trier les stations par degré croissant :

1. **Tri par sélection (Selection Sort)**
2. **Tri par insertion (Insertion Sort)**
3. **Tri à bulles (Bubble Sort)**
4. **Tri fusion (Merge Sort)**
5. **Tri rapide (Quick Sort)**

3.2 Résultats expérimentaux

Tests effectués sur **125 stations** du réseau :

Algorithme	Comparaisons	Échanges	Complexité théorique
Sélection	7 750	124	$O(n^2)$
Insertion	~3 200	~2 100	$O(n^2)$
Bulles	7 750	~4 500	$O(n^2)$
Fusion	~850	0*	$O(n \log n)$
Rapide	~1 100	~380	$O(n \log n)$

*Le tri fusion ne compte pas les déplacements comme des échanges

3.3 Analyse comparative

Algorithmes en $O(n^2)$:

- Le tri par **sélection** fait toujours $n(n-1)/2$ comparaisons mais minimise les échanges
- Le tri par **insertion** est plus efficace sur des données presque triées
- Le tri à **bulles** est le moins efficace avec le maximum d'échanges

Algorithmes en $O(n \log n)$:

- Le tri **fusion** garantit toujours $O(n \log n)$ mais nécessite de la mémoire supplémentaire
- Le tri **rapide** est généralement le plus performant en pratique

3.4 Conclusions

Pour ce projet avec ~125 stations :

1. **Différence notable** : Les algorithmes $O(n \log n)$ font ~7 fois moins de comparaisons
2. **Meilleur choix** : Tri rapide (Quick Sort) - bon compromis entre vitesse et mémoire
3. **Cas particulier** : Si les données sont presque triées, le tri par insertion serait optimal

Sur des réseaux plus grands (>1000 stations), la différence serait encore plus marquée, rendant les tris en $O(n^2)$ impraticables.

4. Algorithme de Dijkstra

4.1 Principe

L'algorithme trouve le chemin le plus court (en temps) entre deux stations :

1. Initialiser toutes les distances à l'infini sauf la source (0)
2. Répéter n fois :
 - Sélectionner le nœud non visité avec la distance minimale
 - Mettre à jour les distances de ses voisins
3. Reconstruire le chemin via les prédécesseurs

4.2 Complexité

- **Complexité temporelle** : $O(n^2)$ avec l'implémentation actuelle

4.3 Exemple de résultat

Station de départ : 0

Station d'arrivée : 24

Temps total: 58 minutes

Chemin: 0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> ... -> 24

5. Conclusion

Ce projet illustre l'importance du choix des structures de données et des algorithmes :

- Les **listes d'adjacence** sont adaptées aux graphes creux
- Les algorithmes de tri en **$O(n \log n)$** sont indispensables pour de grandes données
- L'algorithme de **Dijkstra** permet de résoudre efficacement le problème du plus court chemin